

 ~/estrutura-de-dados/apresentacao

```
$ python quicksort.py
```

```
#  
Loading algorithm...
```

QUICKSORT

```
// Algoritmo de ordenação rápida e eficiente
```

```
team = ["Hugo", "Hyziel", "Luana"]
```

```
subject = "Estrutura de Dados"
```



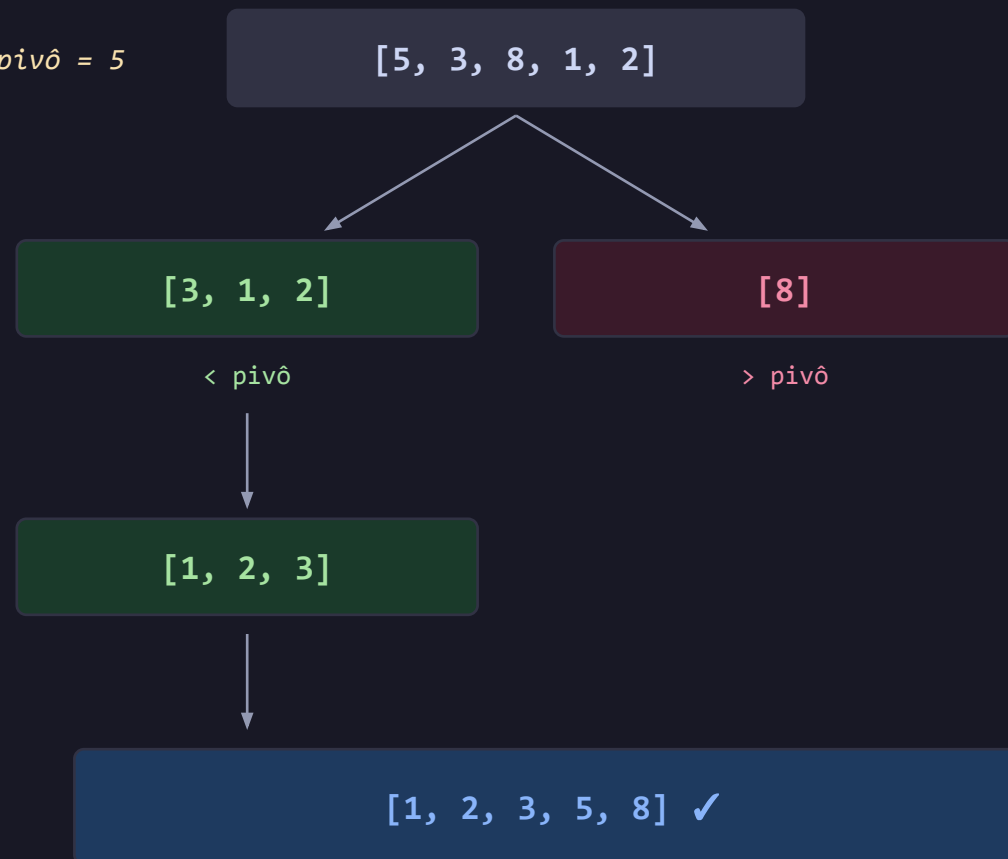
> ideia_geral() Como o QuickSort pensa

Estratégia: Dividir e Conquistar

1. Escolher um **pivô** na lista.
2. Particionar: colocar **menores** à esquerda e **maiores** à direita do pivô.
3. Aplicar recursivamente nas duas sublistas geradas.
4. Quando a lista tiver 0 ou 1 elemento, ela já está ordenada.

// visual

pivô = 5



> passo_a_passo() Ordenando [5, 3, 8, 1, 2]

[1] Lista inicial – escolhemos o primeiro como pivô

lista: [5, 3, 8, 1, 2] pivô = 5

menores → [3, 1, 2] maiores → [8]

[2] Resultado parcial: menores + [5] + maiores

lista: [3, 1, 2] + [5] + [8] pivô = 3

menores → [1, 2] maiores → []

[3] Recursão em [3, 1, 2] – pivô = 3

lista: [3, 1, 2] pivô = 3

menores → [1, 2] maiores → []

[4] Recursão em [1, 2] – pivô = 1

lista: [1, 2] pivô = 1

menores → [] maiores → [2]

[5] Concatenando tudo: [1, 2, 3, 5, 8] ✓ *ordenado*

> implementacao.py Código em Python

```
1 def quicksort(lista):
2     # caso base: lista vazia ou com 1 elemento
3     if len(lista) <= 1:
4         return lista
5
6     pivo = lista[0]
7     menores = [x for x in lista[1:] if x <= pivo]
8     maiores = [x for x in lista[1:] if x > pivo]
9
10    return quicksort(menores) + [pivo] + quicksort(maiores)
11
12 # teste
13 print(quicksort([5, 3, 8, 1, 2])) # → [1, 2, 3, 5, 8]
14
```

> explicar() Estrutura do código

► Caso base

```
if len(lista) <= 1: return lista
```

Uma lista vazia ou com 1 elemento já está ordenada – encerra a recursão.

► Partição (list comprehensions)

```
menores = [x for x in lista[1:] if x <= pivo]  
maiores = [x for x in lista[1:] if x > pivo]
```

Duas listas: menores/iguais ao pivô e maiores que o pivô.

► Escolha do pivô

```
pivo = lista[0]
```

Usamos o primeiro elemento como pivô. Ele será a referência da partição.

► Chamada recursiva

```
return quicksort(menores) + [pivo] + quicksort(maiores)
```

Ordena cada lado e concatena: lado_esq + pivô + lado_dir.

► Por que funciona?

A cada chamada o problema diminui (sublistas menores). O pivô fica em sua posição final correta a cada partição. Quando todas as recursões terminam, a lista está totalmente ordenada.

> complexidade() Análise de tempo e espaço

Melhor caso

$O(n \log n)$

Pivô divide a lista em
duas metades equilibradas.

Pior caso

$O(n^2)$

Lista já ordenada com
pivô fixo nas pontas.

// observações

- ▶ **Espaço:** $O(\log n)$ em média (pilha de recursão), podendo chegar a $O(n^2)$ no pior caso.
- ▶ **Estável?** Não! elementos iguais podem trocar de posição relativa.
- ▶ **In-place?** Sim, na implementação clássica (com troca de índices). Nesta versão didática usamos listas auxiliares.
- ▶ **Dica:** escolher o **pivô** de forma inteligente (mediana, aleatório) evita o pior caso.

> pros_e_contras() Vantagens e Limitações

✓ vantagens

- ▶ Muito rápido na prática – na média, mais rápido que alternativas como *MergeSort* e *HeapSort*.
- ▶ Usado em bibliotecas padrão de várias linguagens.
- ▶ Complexidade média $O(n \log n)$.
- ▶ Pode ser feito *in-place* (pouca memória extra).
- ▶ Implementação relativamente simples: (recursão + partição).
- ▶ Tem excelente desempenho com o cache do processador, pois lida com segmentos contíguos de memória.

✗ limitações

- ▶ Pior caso $O(n^2)$ – pivô mal escolhido em listas já ordenadas.
- ▶ Não é *estável* por padrão.
- ▶ Recursão profunda pode causar *stack overflow* em listas muito grandes.
- ▶ Desempenho depende fortemente da escolha do pivô.
- ▶ Para listas pequenas, algoritmos simples (Insertion Sort) podem ser mais rápidos.

> conclusao() Quando usar e considerações finais

Quando o QuickSort faz mais sentido

- ▶ Conjuntos grandes de dados em memória, onde a velocidade média importa mais que estabilidade.
- ▶ Quando há boa aleatoriedade nos dados (evita o pior caso).
- ▶ Bibliotecas internas: `sorted()` do Python, `Arrays.sort()` do Java, `qsort` em C – usam variantes do QuickSort.

```
$ echo "Obrigado!"  
Obrigado!
```

```
$ exit 0
```

```
// Apresentado por: Hugo • Hyziel • Luana
```

```
Estrutura de Dados – QuickSort
```